Analysis of Evolutionary Algorithms for Optimisation

Shvet Maharaj

12 January 2026

Table of contents

1 Design

1.1 Update procedures

Where noted, a *strictly improving update procedure* will be used alongside the updating equations, defined as:

$$\boldsymbol{x}_{k+1} = \begin{cases} \boldsymbol{v}_{k+1} & \text{if} \quad \mathrm{f}(\boldsymbol{v}_{k+1}) < \mathrm{f}(\boldsymbol{x}_k), \\ \boldsymbol{x}_k & \text{otherwise.} \end{cases}$$

Where $\boldsymbol{x}_k$ is the best performing point at iteration k, and $\boldsymbol{v}_{k+1}$ is the candidate point for the next iteration, $k+1$. This candidate point is what is generated by the updating equations.

Alternatively, the update procedure could be a simple iterative updating procedure, where each updated point, $\boldsymbol{v}_{k+1}$, is used directly by the algorithm in the next iteration (thus, the algorithm updates on $\boldsymbol{v}_k$, instead of $\boldsymbol{x}_k$). This would correspond more so to the deterministic algorithms. The best solution, $\boldsymbol{x}_k$ is simply stored as the algorithm progresses.

1.2 Updating equations

At the core of this exploration of the genetic algorithm, and other methods for optimisation, are the *updating equations*. These define how exactly the optimisation paradigm attempts to iteratively improve a candidate solution, $\boldsymbol{v}$.

Hessian methods

Newton-Raphson

$$\boldsymbol{x}_{k+1} = \boldsymbol{v}_k - \frac{f'(\boldsymbol{v}_k)}{f''(\boldsymbol{v}_k)}$$

Gradient methods

Gradient descent

$$\boldsymbol{v}_{k+1} = \boldsymbol{v}_k - \nu \nabla f(\boldsymbol{v}_k),$$

where ν is the learning rate.

Function value methods

Also known as Direct Search, Pattern Search, or Black-Box Methods. These algorithms do not make use of gradient nor of hessian information. This makes them far easier to apply to any context.

Random search (RS)

The specific version of the random search algorithm is informed by the descriptions given in Zabinsky (2011). Algorithms are described by being 'global' or 'local'.

Pure Random Search (PRS) repeatedly performs a global search with a simply improving update procedure.

The updating equation is defined as,

$$\boldsymbol{v}_{k+1} = \boldsymbol{u}.$$

It is notable that neither of the terms $\boldsymbol{x}_k$ nor $\boldsymbol{v}_k$ — the best or candidate points found by or for iteration k — are present in this formula.

Each generated point is *independent* of any previous point. The major benefit of this is PRS is 'embarrassingly parallelisable'. This is the only optimisation algorithm that will have this property in this paper.

We test also an additional variant of RS, where the global search is either performed solely for the starting point), $\boldsymbol{x}_0$, or skipped entirely in favour of a user-defined starting point. Subsequent points are generated locally from the hypersphere (the '$(N-1)$-sphere', or the 'surface of the N-ball') around $\boldsymbol{x}_k$.

This is Rastrigin's **Fixed Step Size Random Search (FSSRS)**(Rastrigin 1963).

An efficient way to sample uniformly from the surface of an N-ball is given by Marsaglia (1972).

$$\boldsymbol{v}_{k+1} = \boldsymbol{x}_k + h\boldsymbol{u},$$

Here, $\boldsymbol{u}$ is a normalised N-dimensional vector sample from the standard normal distribution such that,

$\boldsymbol{u} = \frac{1}{\|\boldsymbol{z}\|}\boldsymbol{z}$ and $\boldsymbol{z} \sim \mathcal{N}_N(0,1)$, and where h is a step size hyperparameter.

Genetic algorithms

Genetic algorithms are a subset of evolutionary algorithms, typified by the use of all of three genetic operators: crossover, selection, and mutation. A *real-coded* (in reference to $\mathbb{R}$, the real number set) genetic algorithm is used for greater applicability to arbitrary problems and models, despite genetic processes occurring with a discrete set of genes (ACGT) in nature.

Notation does shift here, due to the genetic algorithm necessarily requiring multiple candidates for each iteration. Our previous notation referencing a single candidate per iteration is thus no longer feasible.

We are implementing genetic algorithms using the `GA` package, as described in Scrucca (2013).

A brief explanation of the updating procedure follows.

Fitness evaluation

Fitness for each of the 'genotypes' (candidate solutions), $\theta_i^{(k)}$ is calculated, where i is an arbitrary index of the genotype in the generation k. This is typically the objective function of interest, $\mathrm{f}(\cdot)$.

Selection

A subset of genotypes from the current generation's population becomes selected as a separate population known as the 'reproducing population' Genotypes are selected (with replacement) through a weighted selection process, based typically proportionally on the fitness of each genotype, f $\left(\theta_i^{(k)}\right)$, with probability $p_i^{(k)}$.

An elitist strategy can be used, which means that even if the best performers are unluckily not selected, they still make it to the next population. By default `gareal_lsSelection` is used, which is proportional selection with fitness linear scaling.

Crossover

The core of a genetic algorithm, the crossover operation, brings a means to generate additional 'child' (next generation) genotypes through parts of the 'genetic information' (parameter values) from either of the 'parent' (current generation) genotypes. Alongside mutation, crossover applied on the reproducing population creates the new, next generation population. By default, `gareal_laCrossover` is used, which is local arithmetic crossover.

Mutation

The randomly shifting of the values of 'genes' (parameters) within genotypes. The probability of mutation can be set by the user. Alongside crossover, mutation applied on the reproducing population creates the new, next generation population. By default, `gareal_raMutation` is used, which is uniform random mutation.

Crossover and mutation operations increase diversity in a population, allowing us to explore more of the sample space (exploration), while selection operations decrease diversity in favour of increased fitness (exploitation).

Marsaglia, George. 1972. "Choosing a Point from the Surface of a Sphere." *The Annals of Mathematical Statistics* 43 (2): 645–46. https://doi.org/10.1214/aoms/1177692644.

Rastrigin, L. A. 1963. "The Convergence of the Random Search Method in the Extremal Control of a Many Parameter System." *Automaton & Remote Control* 24: 1337–42. https://cir.nii.ac.jp/crid/1573950398967802624.

Scrucca, Luca. 2013. "**GA**: A Package for Genetic Algorithms in*R*." *Journal of Statistical Software* 53 (4). https://doi.org/10.18637/jss.v053.i04.

Zabinsky, Zelda B. 2011. "Random Search Algorithms." *Wiley Encyclopedia of Operations Research and Management Science*, January. https://doi.org/10.1002/9780470400531.eorms0704.